

Análisis de ingeniería inversa sobre un sistema *legacy*

Base de Datos

20 de junio de 2026

1. Introducción

Se nos ha encomendado realizar ingeniería inversa y posteriormente el análisis de un conjunto de archivos que comprenden la definición estructural y lógica de la Base de Datos de un sistema "Legacy". Con la finalidad de practicar e integrar todo lo aprendido.

2. Descripción del problema

Lo que hace este programa en SQL, es como un archivero digital para organizar los datos de una escuela, y que todo este donde debe estar ya que crea diferentes listas que se conectan entre sí para guardar la información bien estructurada y limpia.

3. Análisis

Pero, ¿Qué es lo que hace exactamente? Bueno en primer lugar tenemos que crear un espacio inicial donde se va a guardar todo

TABLAS	¿QUE ORGANIZA?
Departamentos y Profesores	Hace una lista de las áreas de la escuela y registra a los profesores identificando a qué área pertenece cada uno
Cursos y Aulas	Registra materias que se imparten, qué profesor las imparte y los salones con disponibilidad y cupo máximo
Horarios	Acomoda el día, a qué hora y en qué salón toca cada materia
Estudiantes e Inscripciones	Guarda los datos personales de los alumnos y lleva un control de las materias que tienen inscritas
Calificaciones	Crea un espacio exclusivo para guardar las calificaciones finales de los alumnos en cada materia que tomaron

4. Diagramas y resultados

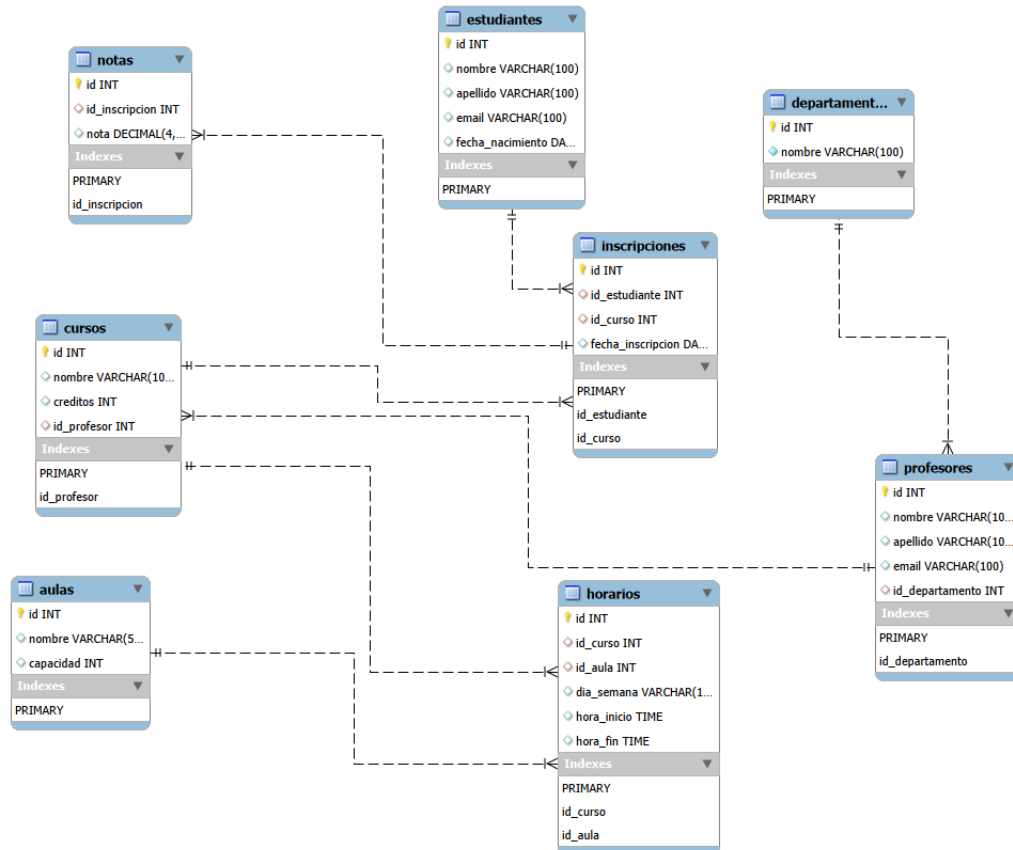
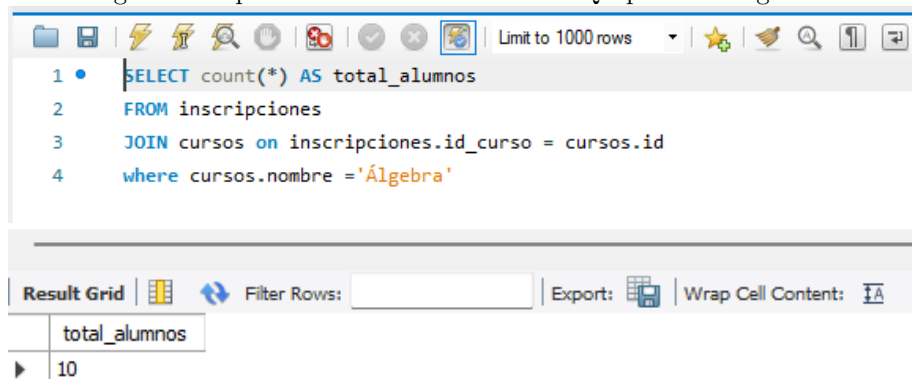


Figura 1: Diagrama Entidad - Relacion del Sistema

5. Implementación y validación

Figura 2: Implementacion de Cosulta en SQL para la Pregunta 1



The screenshot shows a SQL query editor with a toolbar at the top containing icons for file operations, execution, and settings. The query text is as follows:

```
1 • SELECT count(*) AS total_alumnos
2 FROM inscripciones
3 JOIN cursos on inscripciones.id_curso = cursos.id
4 where cursos.nombre = 'Álgebra'
```

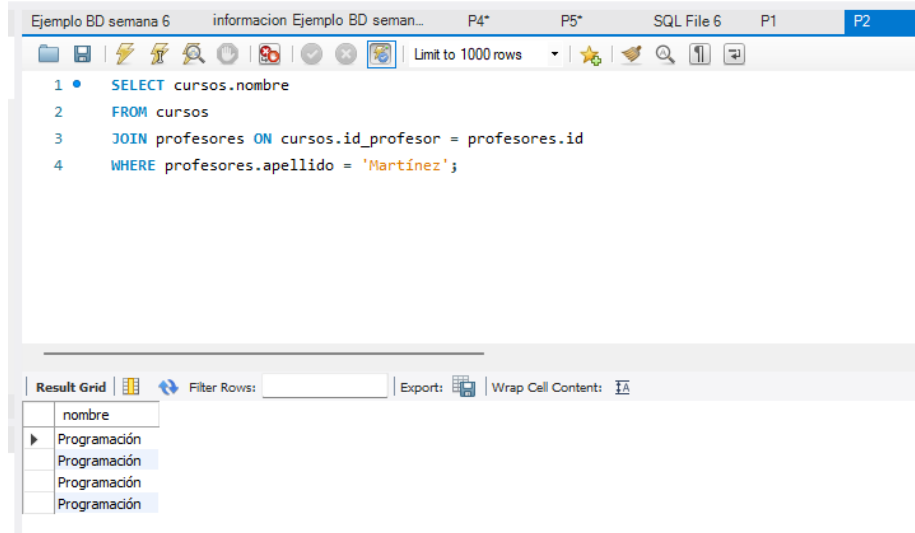
Below the query editor, there is a 'Result Grid' section with a toolbar for filtering, exporting, and wrapping text. The result grid displays the following data:

total_alumnos
10

¿Cuales son los estudiantes inscritos en el curso de Algebra?

Lo que tuve que hacer fue poner el select busca a todos los alumnos , luego se va a la tabla de inscripciones, después busca con un Join los cursos que tengan inscripciones, *id_curso*,y para finalizar busca el nombre del curso que en este caso es Álgebra.

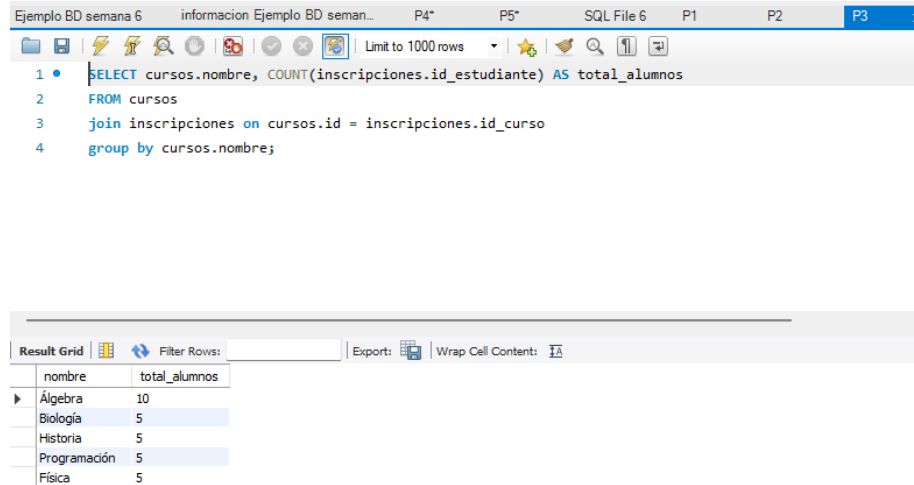
Figura 3: Implementacion de Consulta en SQL para la Pregunta 2



¿Que cursos imparte el Profesor Martinez?

Programacion , de igual manera ocupamos el JOIN para conectarnos a la tabla de cursos con la de profesores con el ON es como un puente que une los datos con el ID del profesor

Figura 4: Implementación de Consulta en SQL para la Pregunta 3



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and settings. The query editor contains the following SQL code:

```

1 • SELECT cursos.nombre, COUNT(inscripciones.id_estudiante) AS total_alumnos
2 FROM cursos
3 JOIN inscripciones ON cursos.id = inscripciones.id_curso
4 GROUP BY cursos.nombre;

```

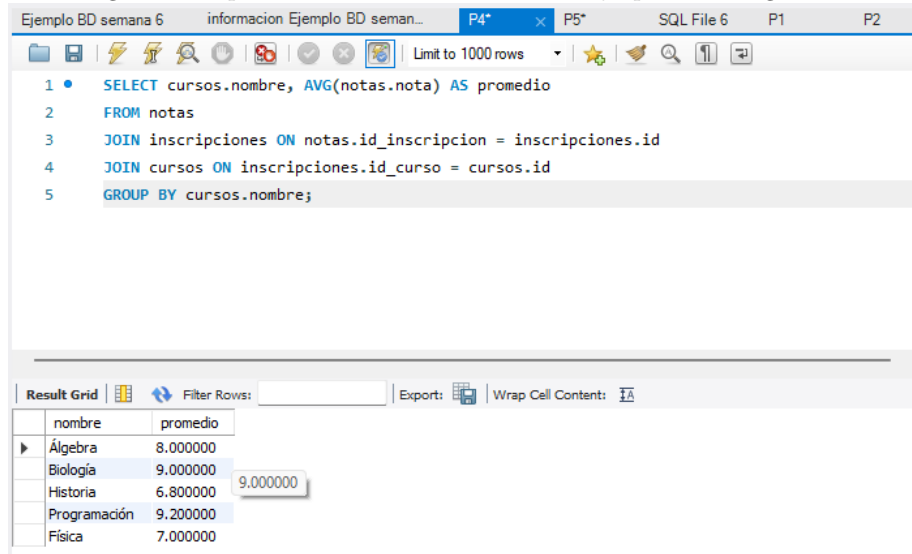
Below the query editor, the 'Result Grid' tab is active, displaying the results of the query in a table format. The table has two columns: 'nombre' and 'total_alumnos'.

nombre	total_alumnos
Álgebra	10
Biología	5
Historia	5
Programación	5
Física	5

¿Cuántos estudiantes están inscritos en cada curso?

Álgebra = 6, Biología = 3, Historia = 3, Programación = 3, Física = 3
 ocupe un Count, que es un contador, que le dice a la BD que sume todos los alumnos, y el GROUP BY es la que ordena a la BD que agrupe todos los resultados guiándose por el nombre de la materia

Figura 5: Implementación de Consulta en SQL para la Pregunta 4



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```

1 • SELECT cursos.nombre, AVG(notas.nota) AS promedio
2 FROM notas
3 JOIN inscripciones ON notas.id_inscripcion = inscripciones.id
4 JOIN cursos ON inscripciones.id_curso = cursos.id
5 GROUP BY cursos.nombre;

```

The result grid displays the following data:

nombre	promedio
Álgebra	8.000000
Biología	9.000000
Historia	6.800000
Programación	9.200000
Física	7.000000

¿Cual es el promedio de notas por curso?

Biología con un promedio de 9.0 ponemos un select de cursos.nombre y ponemos una nueva función AVG (notas.nota) que le dice a BD que quiere el promedio de las notas de la tabla de notas pero hazlas como Promedio hace 2 JOIN que revisa cursos e inscripciones y los agrupa por Cursos.nombre

Figura 6: Implementación de Consulta en SQL para la Pregunta 5

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and search. The query editor contains the following SQL code:

```

1 • SELECT estudiantes.nombre, estudiantes.apellido, notas.nota
2 FROM estudiantes
3 JOIN inscripciones ON estudiantes.id = inscripciones.id_estudiante
4 JOIN notas ON inscripciones.id = notas.id_inscripcion
5 WHERE notas.nota > 8.0;

```

Below the query editor, the 'Result Grid' tab is active, displaying the results of the query. The results are shown in a table with columns 'nombre', 'apellido', and 'nota'.

nombre	apellido	nota
Juan	Pérez	8.50
Juan	Pérez	9.00
Carlos	Sánchez	9.20
Juan	Pérez	8.50
Juan	Pérez	9.00
Carlos	Sánchez	9.20
Juan	Pérez	8.50
Juan	Pérez	9.00
Carlos	Sánchez	9.20
Juan	Pérez	8.50
Juan	Pérez	9.00
Carlos	Sánchez	9.20
Juan	Pérez	8.50
Juan	Pérez	9.00
Carlos	Sánchez	9.20

¿Qué lista de estudiantes tiene notas mayores a 8.0?

Juan Pérez 8.50 Juan Pérez 9.00 Carlos Sánchez 9.20 Juan Pérez 8.50 Juan Pérez 9.00 Carlos Sánchez 9.20 Juan Pérez 8.50 Juan Pérez 9.00 Carlos Sánchez 9.20

hacemos nuevamente un select que pida nombre , apellidos y notas de todos los alumnos inscritos a un curso , luego hacemos unos JOINS , el primero pide las inscripciones x estudiante y que devuelva las inscripciones con el id de estudiante , el segundo las notas de las inscripciones (id) y un where donde se estipule que las solamente nos de los estudiantes con las notas mayores a 8.0

Figura 7: Implementación de Consulta en SQL para la Pregunta 6

```

1 • select cursos.nombre, COUNT(inscripciones.id) AS total_inscripciones
2   FROM cursos
3  JOIN inscripciones ON cursos.id = inscripciones.id_curso
4  GROUP BY cursos.nombre
5  HAVING COUNT(inscripciones.id) > 1;
6

```

nombre	total_inscripciones
Álgebra	10
Biología	5
Historia	5
Programación	5
Física	5

¿Que cursos tienen mas de una inscripcion?

Tenemos que irnos a la tabla de nombre y contar los IDs de todos los curso de nuestra BD , despues tenemos un JOIN para ver el estado de las inscripciones y luego con el ON esta para ver los ids de los cursos y ya para finalizar tenemos un HAVING para saber que cursos tengan más de un inscripción

Figura 8: Implementación de Consulta en SQL para la Pregunta 7

The screenshot shows a SQL query editor interface. At the top, there are tabs labeled 'Ejemplo BD semana 6', 'informacion Ejemplo BD seman...', 'P7', 'P8', and 'P9'. The 'P7' tab is active. Below the tabs is a toolbar with various icons for file operations, execution, and settings. A dropdown menu shows 'Limit to 1000 rows'. The main area contains a SQL query:

```
1 • SELECT profesores.nombre, profesores.apellido
2 FROM profesores
3 JOIN departamentos ON profesores.id_departamento = departamentos.id_departamento
4 WHERE departamentos.nombre = 'Ingeniería';
```

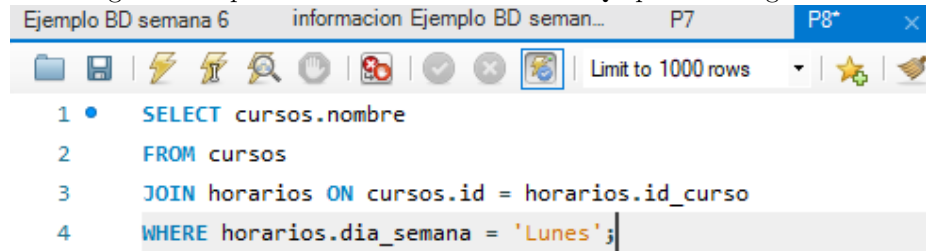
Below the query editor, there is a 'Result Grid' section. It includes a 'Filter Rows:' input field, an 'Export:' button, and a 'Wrap Cell Content:' checkbox. The result grid displays a table with two columns: 'nombre' and 'apellido'. The table contains five rows, all with the name 'Pedro' and the surname 'Martínez'.

	nombre	apellido
▶	Pedro	Martínez
	Pedro	Martínez
	Pedro	Martínez
	Pedro	Martínez
	Pedro	Martínez

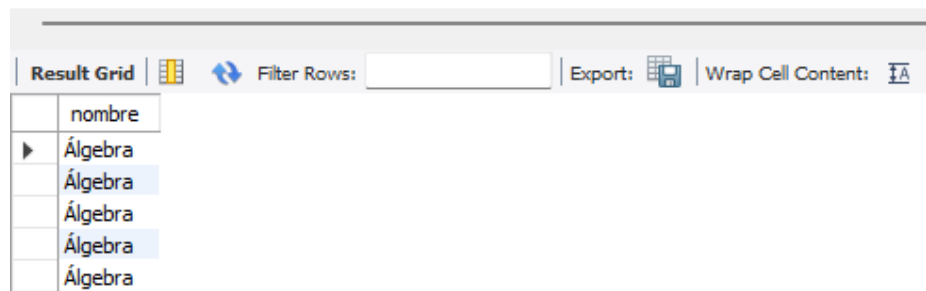
¿Que profesores pertenecen al departamento de Ingeniería?

Seleccionamos los nombres y apellidos de los profesores, donde este el id.departamento selecciona los IDs y ponemos el parametro de que solo queremos a profesores que pertenezcan al departamento de Ingeniería

Figura 9: Implementación de Consulta en SQL para la Pregunta 8



```
1 • SELECT cursos.nombre
2 FROM cursos
3 JOIN horarios ON cursos.id = horarios.id_curso
4 WHERE horarios.dia_semana = 'Lunes';
```

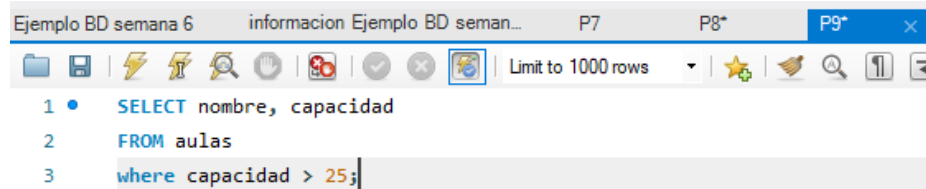


	nombre
▶	Álgebra
	Álgebra
	Álgebra
	Álgebra
	Álgebra

¿Cuales son los cursos programados los Lunes?

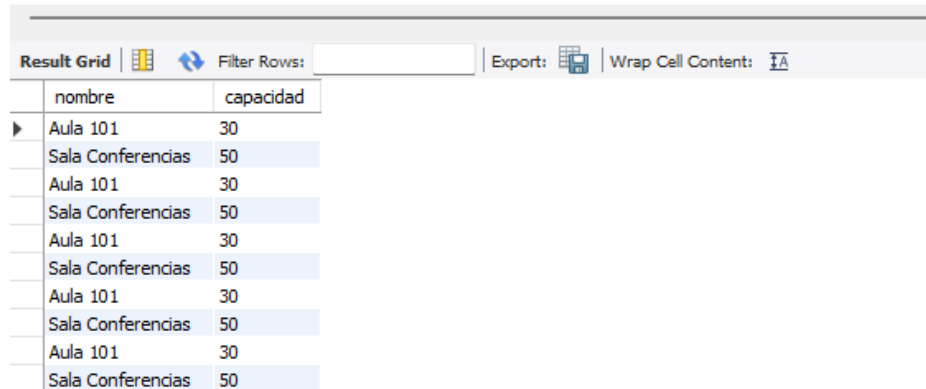
Seleccionamos de las tablas/columnas (nombres, hora_inicio, c_id) desde la tabla de horario, y unimos (join) en el horario de id del curso donde solo este programado el día Lunes

Figura 10: Implementación de Consulta en SQL para la Pregunta 9



```

1 • SELECT nombre, capacidad
2 FROM aulas
3 where capacidad > 25;
  
```



	nombre	capacidad
▶	Aula 101	30
	Sala Conferencias	50
	Aula 101	30
	Sala Conferencias	50
	Aula 101	30
	Sala Conferencias	50
	Aula 101	30
	Sala Conferencias	50
	Aula 101	30
	Sala Conferencias	50

¿Que aulas tienen una capacidad mayor a 25?

Seleccionamos el nombre y la capacidad desde la tabla de Aulas donde la capacidad sea mayor a 25 alumnos.

Ejemplo BD semana 6 informacion Ejemplo BD seman... P10 x

```
1 • SELECT cursos.nombre AS curso, aulas.nombre AS aul
2 FROM cursos
3 JOIN horarios ON cursos.id = horarios.id_curso
4 JOIN aulas ON horarios.id_aula = aulas.id;
```

	curso	aula	dia_semana	hora_inicio
▶	Álgebra	Aula 101	Lunes	08:00:00
	Biología	Aula 102	Martes	10:00:00
	Historia	Sala Conferencias	Miércoles	13:00:00
	Programación	Laboratorio 1	Jueves	09:00:00
	Física	Aula 102	Viernes	08:00:00
	Álgebra	Aula 101	Lunes	08:00:00
	Biología	Aula 102	Martes	10:00:00
	Historia	Sala Conferencias	Miércoles	13:00:00
	Programación	Laboratorio 1	Jueves	09:00:00
	Física	Aula 102	Viernes	08:00:00
	Álgebra	Aula 101	Lunes	08:00:00
	Biología	Aula 102	Martes	10:00:00
	Historia	Sala Conferencias	Miércoles	13:00:00
	Programación	Laboratorio 1	Jueves	09:00:00
	Física	Aula 102	Viernes	08:00:00
	Álgebra	Aula 101	Lunes	08:00:00
	Biología	Aula 102	Martes	10:00:00
	Historia	Sala Conferencias	Miércoles	13:00:00
	Programación	Laboratorio 1	Jueves	09:00:00
	Física	Aula 102	Viernes	08:00:00
	Álgebra	Aula 101	Lunes	08:00:00
	Biología	Aula 102	Martes	10:00:00
	Historia	Sala Conferencias	Miércoles	13:00:00
	Programación	Laboratorio 1	Jueves	09:00:00

Figura 12: Implementación de Consulta en SQL para la Pregunta 11

The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
3 JOIN inscripciones i ON n.id_inscripcion = i.id
4 JOIN estudiantes e ON i.id_estudiante = e.id
5 JOIN cursos c ON i.id_curso = c.id
6 ORDER BY n.nota DESC
7 LIMIT 1;
8
```

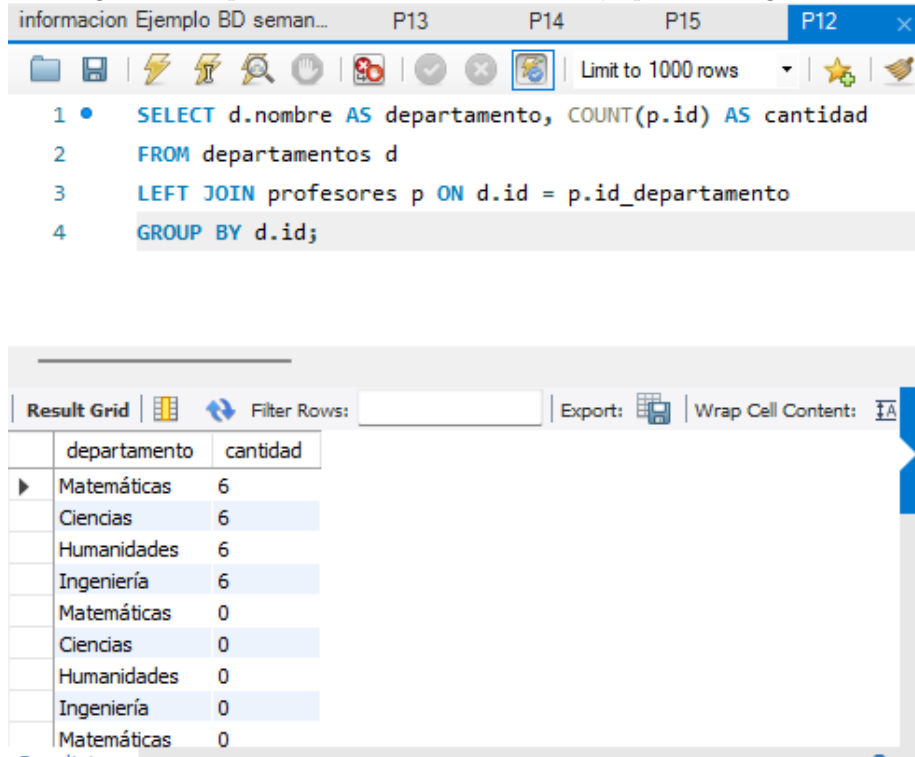
Below the query editor is the 'Result Grid' section, which includes a 'Filter Rows' input, an 'Export' button, a 'Wrap Cell Content' checkbox, and a 'Fetch rows' button. The result grid displays the following data:

	nombre	apellido	curso	nota
►	Carlos	Sánchez	Programación	9.20

Lista de cursos con su aula y horario

Seleccionamos de la tabla de estudiantes sus nombres y apellidos, y el nombre del curso (de la tabla de cursos) va a ser seleccionado como un curso y de la tabla notas vamos a consultar la notas de todos los cursos. Después vamos a implementar 3 JOINS (que van a solicitar/consultar inscripciones, estudiantes y cursos) y va a ordenar las notas mayores solamente y va a relacionarla con el curso.

Figura 13: Implementación de Consulta en SQL para la Pregunta 12



The screenshot shows a SQL query editor with the following query:

```

1 • SELECT d.nombre AS departamento, COUNT(p.id) AS cantidad
2 FROM departamentos d
3 LEFT JOIN profesores p ON d.id = p.id_departamento
4 GROUP BY d.id;

```

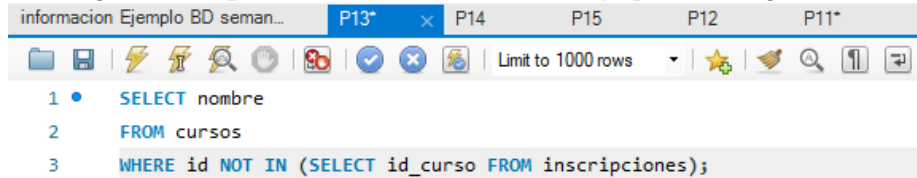
Below the query, the results are displayed in a table:

departamento	cantidad
Matemáticas	6
Ciencias	6
Humanidades	6
Ingeniería	6
Matemáticas	0
Ciencias	0
Humanidades	0
Ingeniería	0
Matemáticas	0

¿Cuántos profesores hay por departamento?

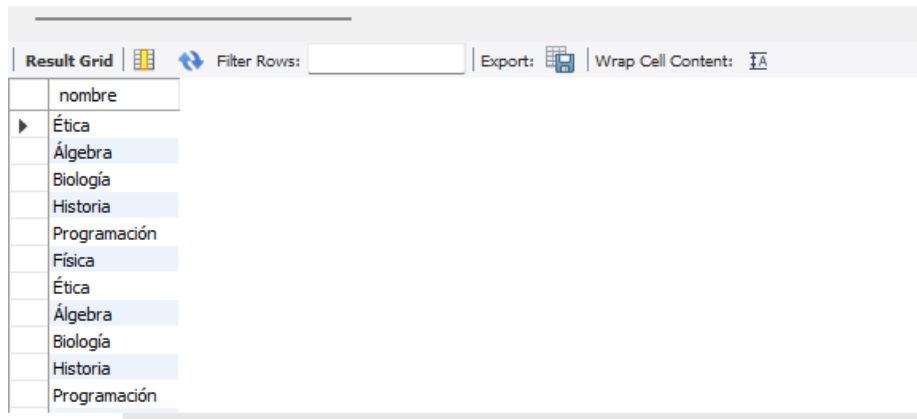
Seleccionamos de la tabla de departamentos con un `COUNT` la cantidad de profesores ocupamos un `LEFT JOIN` que junte a profesores con su id de departamento y los juntamos con el id de departamento de la tabla de departamentos y para finalizar agrupamos por el nombre del departamento

Figura 14: Implementación de Consulta en SQL para la Pregunta 13



```

1 • SELECT nombre
2 FROM cursos
3 WHERE id NOT IN (SELECT id_curso FROM inscripciones);
  
```

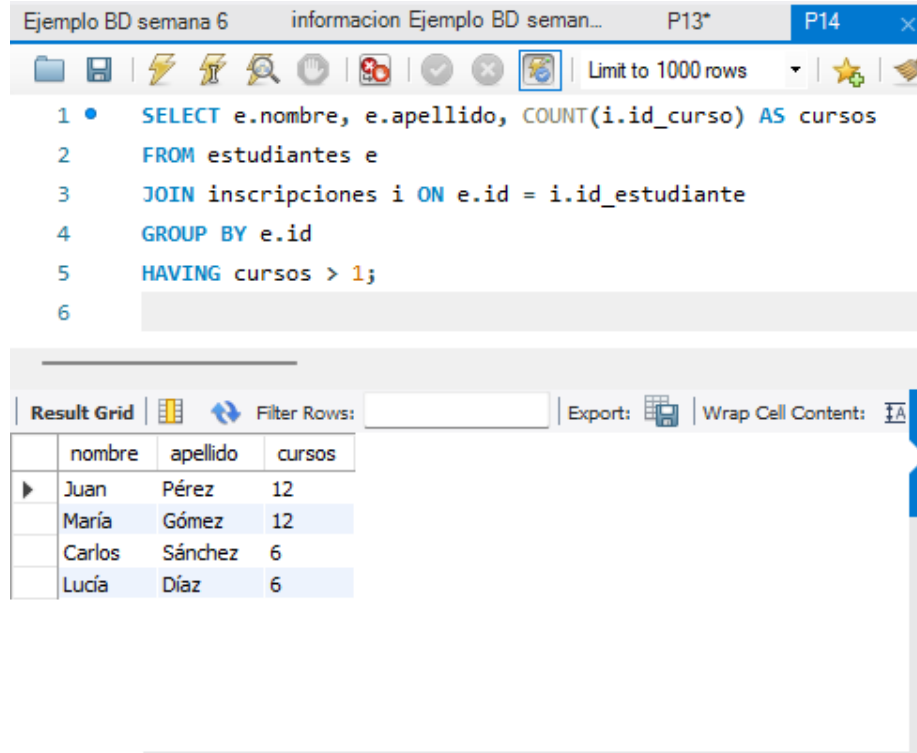


	nombre
▶	Ética
	Álgebra
	Biología
	Historia
	Programación
	Física
	Ética
	Álgebra
	Biología
	Historia
	Programación

¿Cuántos profesores hay por departamento?

Aquí seleccionamos el nombre del curso y el nombre del profesor, luego hacemos un JOIN para unir la tabla de cursos con la de profesores y el ON es para unir los id del profesor con el id del curso y para finalizar ordenamos por el nombre del curso

Figura 15: Implementación de Consulta en SQL para la Pregunta 14



1 • **SELECT** e.nombre, e.apellido, COUNT(i.id_curso) AS cursos
2 **FROM** estudiantes e
3 **JOIN** inscripciones i **ON** e.id = i.id_estudiante
4 **GROUP BY** e.id
5 **HAVING** cursos > 1;
6

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	nombre	apellido	cursos
▶	Juan	Pérez	12
	María	Gómez	12
	Carlos	Sánchez	6
	Lucía	Díaz	6

¿Que cursos no tienen estudiantes inscritos?

Seleccionamos el nombre del curso, luego hacemos un LEFT JOIN para unir la tabla de cursos con la de inscripciones

Figura 16: Implementación de Consulta en SQL para la Pregunta 15

The screenshot shows a SQL query editor with the following query:

```

1 • SELECT c.nombre, AVG(n.nota) AS promedio
2   FROM cursos c
3  JOIN inscripciones i ON c.id = i.id_curso
4  JOIN notas n ON i.id = n.id_inscripcion
5  GROUP BY c.id
6  ORDER BY promedio ASC
7  LIMIT 1;

```

Below the query editor, the result grid is displayed with the following data:

	nombre	promedio
▶	Historia	6.800000

¿Que estudiantes estan inscritos en mas de un curso?

Seleccionamos el nombre y apellido de los estudiantes, luego hacemos un JOIN

6. Conclusiones

En conclusión , este ejercicio de ingeniería inversa nos permitió comprender a fondo la estructura y lógica de una base de datos "legacy". A través del análisis de los archivos proporcionados, pudimos identificar las tablas, relaciones y consultas necesarias para extraer información relevante.